

Plan de Branching – SmartLogix

Estrategia Git Flow

1. Descripción General

SmartLogix utiliza **Git Flow** como estrategia de control de versiones. Esta metodología organiza el desarrollo en ramas con propósitos específicos, garantizando que solo el código verificado y probado llegue a producción.

2. Estructura de Ramas

```
main
  ■■■ develop
    ■■■ feature/ms-inventario
    ■■■ feature/ms-pedidos
    ■■■ feature/ms-envios
    ■■■ feature/bff
    ■■■ feature/frontend
```

3. Descripción de Ramas

Rama	Propósito	Reglas
main	Código en producción, estable	Solo merge desde release/* o hotfix/*. Protegida con revisión obligatoria.
develop	Integración continua de funcionalidades	Rama base para todas las features. Siempre debe compilar.
feature/*	Desarrollo de cada funcionalidad	Se crea desde develop, se mergea de vuelta a develop vía Pull Request.
release/*	Preparación de una versión para producción	Se crea desde develop cuando hay funcionalidades completas. Solo se permiten bugfixes.
hotfix/*	Correcciones urgentes en producción	Se crea desde main, se mergea tanto a main como a develop.

4. Flujo de Trabajo por Integrante

Integrante 1 – Ricardo Novoa

Responsable de: ms-inventario + BFF

```
# 1. Crea rama desde develop
git checkout develop
git pull origin develop
git checkout -b feature/ms-inventario

# 2. Desarrolla y commitea frecuentemente
git add .
git commit -m "feat(inventario): agrega entidad Producto con validación JPA"
git commit -m "feat(inventario): implementa Repository Pattern en ProductoRepository"
git commit -m "test(inventario): agrega pruebas unitarias InventarioServiceTest"

# 3. Sube la rama
git push origin feature/ms-inventario

# 4. Abre Pull Request hacia develop
# (revisión por al menos 1 compañero antes de mergear)

# 5. Merge y limpieza
```

```
git checkout develop
git merge --no-ff feature/ms-inventario
git branch -d feature/ms-inventario
```

Integrante 2 – Cristobal Pérez

****Responsable de:**** ms-pedidos + ms-envios

```
git checkout develop && git pull
git checkout -b feature/ms-pedidos
# ... commits ...
git push origin feature/ms-pedidos
# Pull Request → develop
```

Integrante 3 – Benjamín Meneses

****Responsable de:**** Frontend React (todas las páginas)

```
git checkout develop && git pull
git checkout -b feature/frontend-dashboard
# ... commits ...
git push origin feature/frontend-dashboard
# Pull Request → develop
```

5. Convención de Commits (Conventional Commits)

<tipo>(<scope>): <descripción corta>

Tipo	Uso
feat	Nueva funcionalidad
fix	Corrección de bug
test	Añadir o corregir tests
docs	Cambios en documentación
refactor	Refactorización sin cambio de comportamiento
chore	Tareas de configuración (pom.xml, package.json)

****Ejemplos:****

```
feat(pedidos): implementa Factory Method para estados de pedido
fix(inventario): corrige validación de stock negativo en JPA
test(bff): agrega pruebas unitarias BffServiceTest
feat(frontend): agrega página Inventario con CRUD completo
chore(bff): configura puerto 8080 en application.properties
```

6. Resolución de Conflictos

Cuando dos ramas modifican el mismo archivo:

```
# 1. Actualizar develop local
git checkout develop
git pull origin develop

# 2. Hacer rebase de la feature
git checkout feature/mi-feature
git rebase develop

# 3. Si hay conflicto, el git marcará los archivos:
# <<<<<<< HEAD (tu versión)
# =====
# >>>>>>> develop (versión de develop)

# 4. Editar el archivo, resolver manualmente y marcar como resuelto:
git add archivo-resuelto.java
git rebase --continue

# 5. Subir rama actualizada (force-push seguro en feature branches)
git push origin feature/mi-feature --force-with-lease
```

7. Ciclo Release

```
# Cuando develop está estable para lanzar v1.0.0:
git checkout develop
git checkout -b release/v1.0.0

# Solo bugfixes en esta rama
git commit -m "fix(release): corrige configuración de CORS en BFF"

# Merge a main con tag
git checkout main
git merge --no-ff release/v1.0.0
git tag -a v1.0.0 -m "Release SmartLogix v1.0.0"
git push origin main --tags

# Merge de vuelta a develop
git checkout develop
git merge --no-ff release/v1.0.0
git branch -d release/v1.0.0
```

8. Reglas del Equipo

- ■ ****Mínimo 1 revisión**** (code review) antes de mergear a develop
- ■ ****Los tests deben pasar**** antes de abrir un Pull Request
- ■ ****Commits pequeños y frecuentes**** – nunca acumular días de trabajo en un solo commit
- ■ ****Nunca hacer push directo a main**** – siempre mediante Pull Request
- ■ ****Nombres de ramas descriptivos**** – feature/ms-inventario, no feature/test

9. Historial de Merges Ejemplo

```
* a1b2c3d (HEAD -> main, tag: v1.0.0) Merge release/v1.0.0
|\
| * d4e5f6g fix(release): corrige CORS en BFF
| * h7i8j9k Merge feature/frontend-envios -> develop
| * l0m1n2o Merge feature/bff -> develop
| * p3q4r5s Merge feature/ms-envios -> develop
| * t6u7v8w Merge feature/ms-pedidos -> develop
| * x9y0z1a Merge feature/ms-inventario -> develop
|/
* b2c3d4e chore: inicializa repositorio con estructura base
```